



Democratic and Popular Republic of Algeria
Ministry of Higher Education and Scientific Research
Houari Boumediene University of Science and Technology
U.S.T.H.B.

Report of:

Multimedia Mini Project:
Simplified MPEG-4 Video Encoder Pipeline

Performed by:

- **Miss BOUCHEKIMA Anissa** id: 212133039463
- **Miss LOUNIS Yasmine** id: 222231472410

M1 IL / Group 03

1. Introduction and Objectives :

Video compression serves as the essential foundation of modern multimedia systems, converting massive raw visual data into compact, transmittable bitstreams. To explore these mechanisms, this project implements a complete, simplified MPEG-4-like encoder/decoder pipeline in Python that compresses a sequence of source frames into a single binary file (.bin) and fully decodes them back. The performance of this end-to-end architecture is thoroughly measured and evaluated through quantitative experiments analyzing the compression ratio against both the quantization factor and Group of Pictures (GOP) size.

2. Pipeline Description :

a. Part 1 : Pre-processing.

- Each input BGR frame is first converted to the YCbCr color space using `cv2.cvtColor(frame, cv2.COLOR_BGR2YCrCb)`.
- The image is then split into three separate channels:
 - Y (luminance).
 - Cb (blue chrominance).
 - Cr (red chrominance).
- Finally, 4:2:0 chroma subsampling is applied: the Cb and Cr channels are reduced by a factor of 2 in both dimensions using array slicing `Cb[:, ::2, ::2]`.

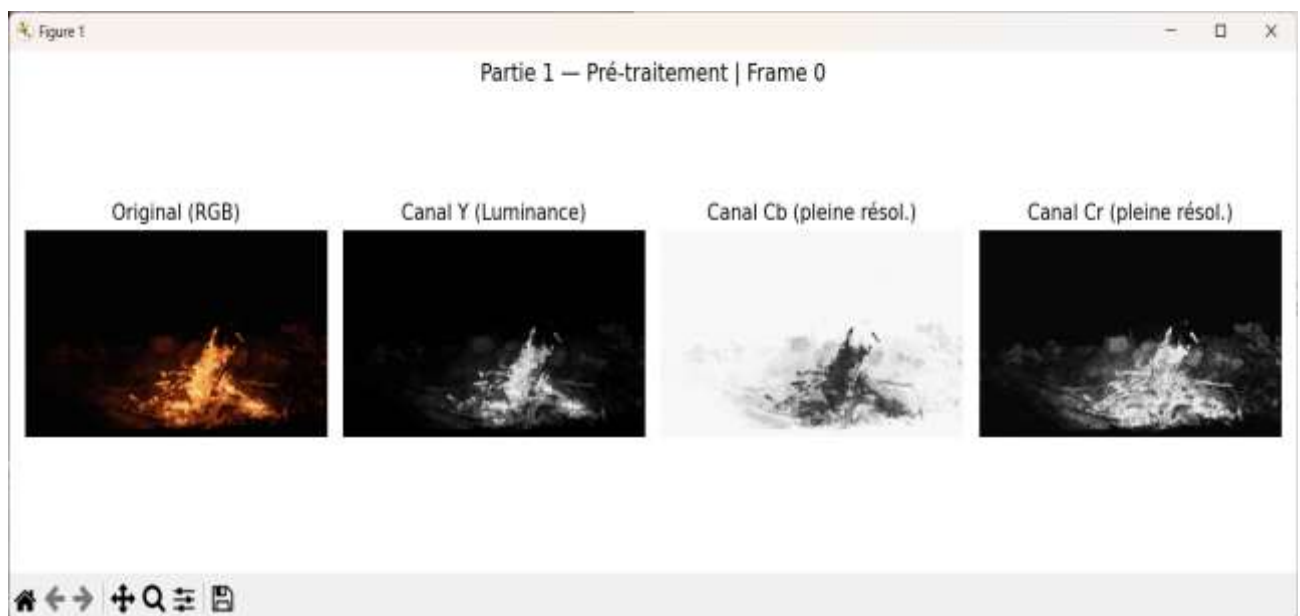


Figure 1 — Part 1 output: original frame and the three separated YCbCr channels.

b. Part 2 : Intra-frame Coding (I-frames):

An I-frame (Intra-frame) is a frame compressed entirely independently, without reference to other frames. The same technique used in JPEG compression is applied:

- **8×8 Block Partitioning:** The Y channel is divided into small blocks of 64 pixels.
- **2D DCT (Discrete Cosine Transform):** Converts pixel values into frequency components. Low frequencies (representing main structures and smooth areas) are concentrated in the upper-left corner.
- **Quantization:** The coefficients are divided by a quantization matrix Q . High-frequency components (fine details) often become zero, which is where information loss occurs.
- **Decoding:** The coefficients are multiplied again by Q (dequantization), followed by the application of the inverse DCT (IDCT).

The quantization factor F_q controls the compression quality:

- a high F_q value results in higher compression but lower quality.

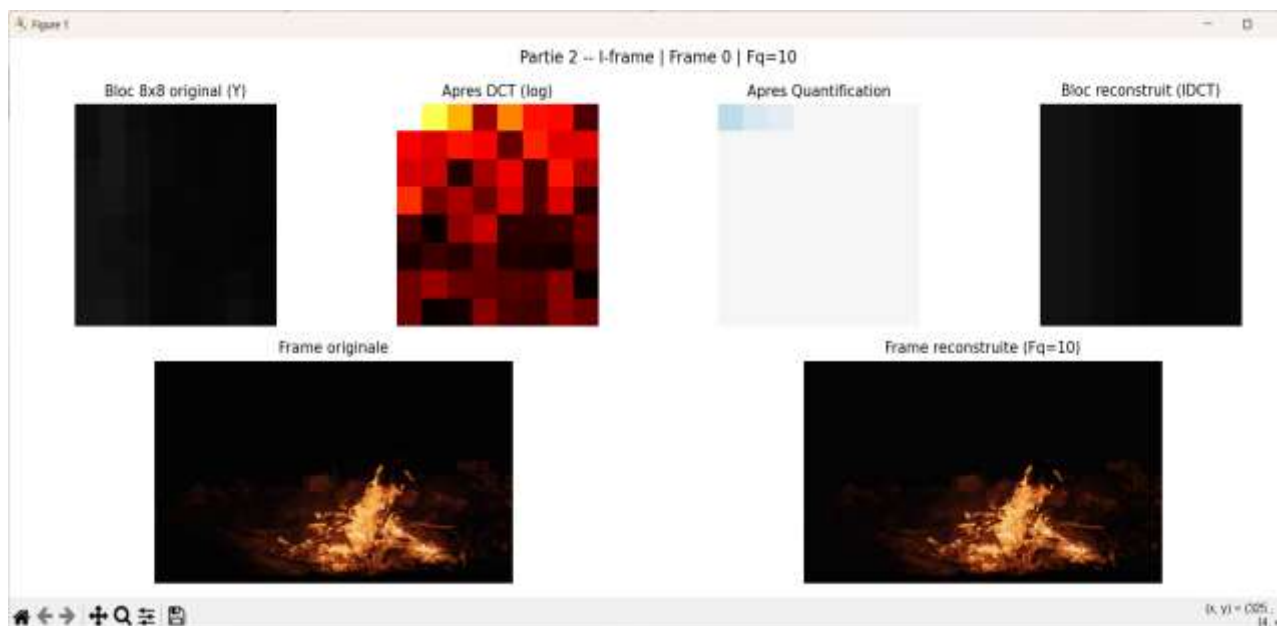


Figure 2 — Part 2 output: an 8×8 block going through the full DCT pipeline and the complete frame original vs reconstructed at $F_q=10$.

c. Part 3 : Inter-frame Coding (P-frames) :

P-frames exploit temporal redundancy between consecutive frames.

- For each one, the Y channel is divided into 16×16 macroblocks each matched against the reference within a search window of ±5 pixels.
- The resulting motion vector (dy, dx) is stored.
- The residual is then computed and compressed with DCT + quantisation.
- The Cb and Cr channels are encoded as I-frame channels independently.
- The decoder reconstructs each P-frame by combining the motion-compensated prediction from the reference frame with the decoded residual:

$$Y_{\text{reconstructed}} = \text{Prediction} + \text{Residual}.$$

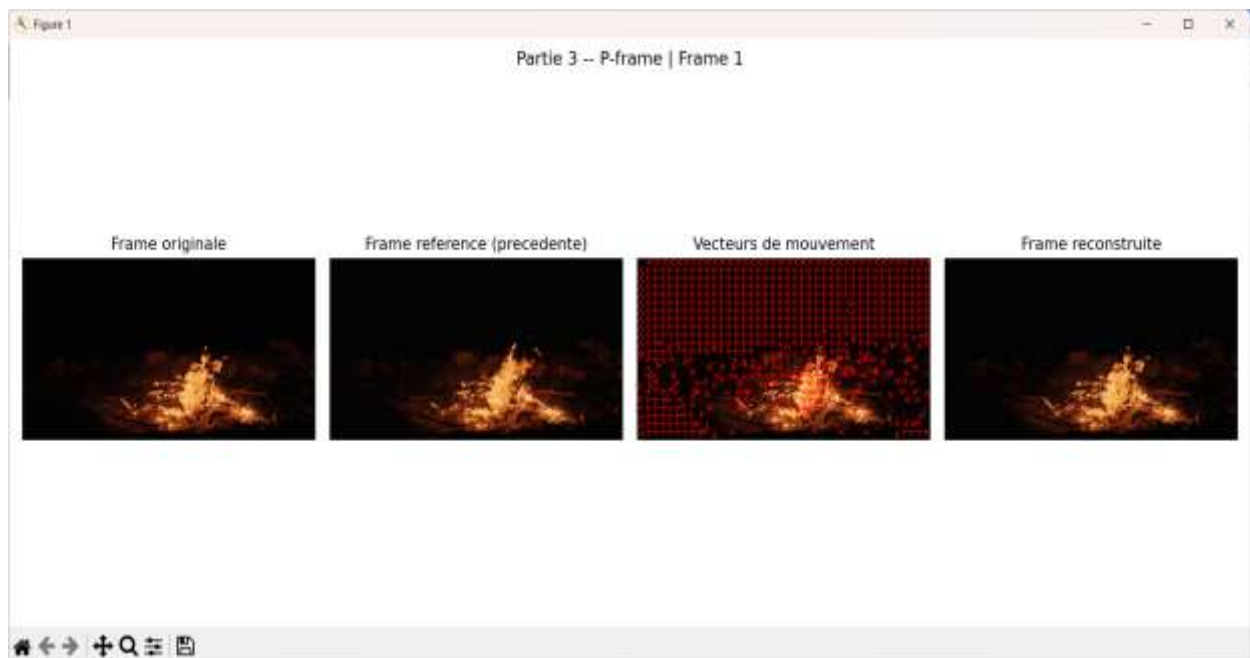


Figure 3 — Part 3 output :motion vectors overlaid on a P-frame (red arrows show displacement), the reconstructed P-frame.

d. Part 4 : Entropy Coding :

The entire encoded video structure (list of I-frame and P-frame dictionaries) is serialized , compressed losslessly and written to a .bin file .

e. Part 5 : Evaluation & Visualisation :

5a — Quality Metrics:

- Compression ratio = $\frac{\text{original size (bytes)}}{\text{.bin file size (bytes)}}$
- Frame-type breakdown: count of I-frames vs P-frames

5b — Pipeline Visualisation:

Visualization of every stage of the pipeline in one figure:

- Row 1: Original input frames with their type label (I or P).
- Row 2: YCbCr channel decomposition (Y, Cb, Cr) of frame 0.
- Row 3: 8×8 block DCT pipeline — raw pixels → DCT → quantised → reconstructed.
- Row 4: Motion vectors overlaid on a P-frame + reconstructed P-frame.
- Row 5: Spatial residual map (pixel-domain difference between current frame and prediction, obtained by decoding the residual and subtracting 128) + reconstructed I-frame.

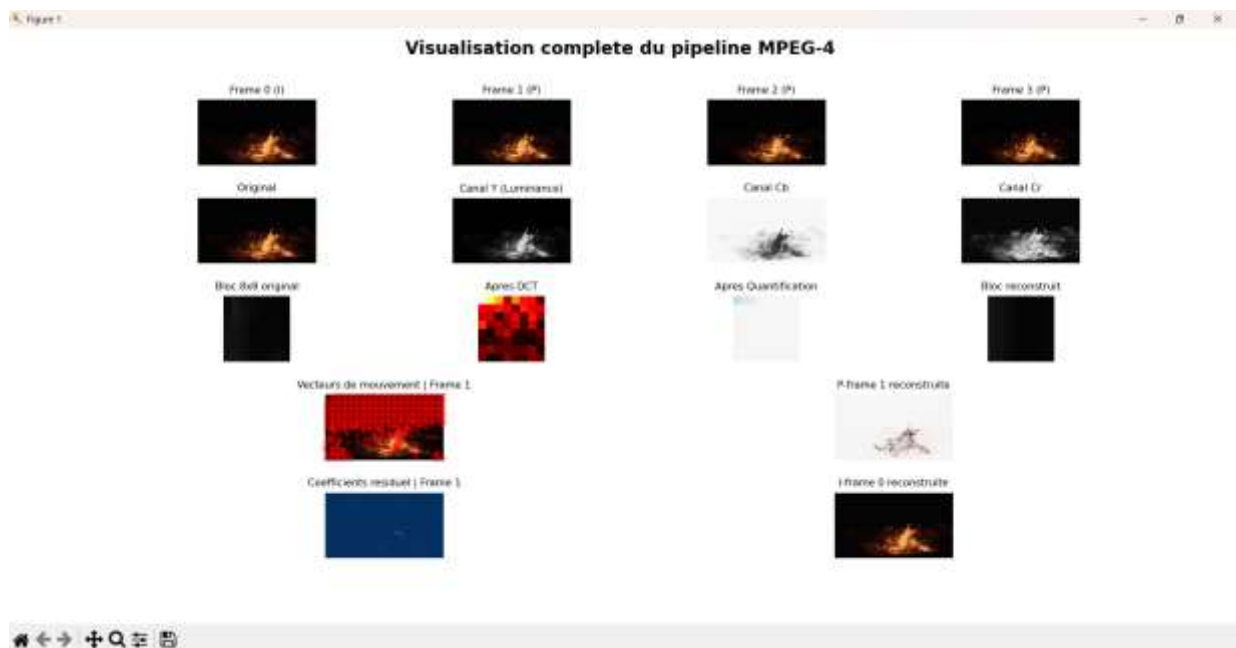


Figure 4 — Part 5b: Complete pipeline visualisation figure covering all 5 stages (original frames, YCbCr channels, DCT block pipeline, motion vectors, spatial residual map).

3. Justification of Design Choices :

a. YCbCr Color Space:

YCbCr was chosen because it separates luminance (Y) from chrominance (Cb, Cr). Human vision is more sensitive to brightness details than color information, making chroma compression possible with limited visual quality loss.

b. 4:2:0 Chroma Subsampling:

The 4:2:0 subsampling reduces the resolution of Cb and Cr channels by keeping one chroma sample for every 2×2 pixel block. This significantly decreases data size while preserving acceptable visual quality.

c. Frame Resizing to 640×360:

Frames larger than 640×360 are resized to reduce computational complexity and memory usage. This resolution provides a good balance between visual quality and processing speed for the project.

d. Block Size of 8×8 for DCT:

An 8×8 block size was used because it is the standard size in JPEG and MPEG compression. It offers a good compromise between compression efficiency and computational cost.

e. Discrete Cosine Transform (DCT):

DCT converts spatial pixel values into frequency coefficients. Most image energy becomes concentrated in low frequencies, allowing high-frequency details to be reduced during quantization.

f. Quantization Matrix:

The standard JPEG quantization matrix was adopted because it prioritizes low-frequency information while aggressively compressing high frequencies. This improves compression with limited perceptual degradation.

g. Padding Strategy:

Padding ensures that frame dimensions are divisible by 8 before DCT processing. This avoids incomplete blocks and guarantees consistent block-based operations across the entire image.

h. int16 Storage for Coefficients:

DCT coefficients were stored using signed 16-bit integers (int16) because frequency coefficients can be negative. This reduces memory usage while preserving sufficient numeric precision.

i. 16×16 Macroblocks:

Motion estimation uses 16×16 macroblocks because this size is commonly used in MPEG standards. It reduces computational cost while still capturing significant motion information.

j. **zlib Entropy Compression:**

zlib was used for entropy coding because it provides efficient lossless compression. It further reduces the size of serialized encoded data.

k. **Pickle Serialization:**

pickle was selected to serialize Python dictionaries and arrays into binary format. This simplifies storage and reconstruction of encoded video structures.

l. **Modular Project Structure:**

The project was divided into separate modules (preprocessing, I-frame, P-frame, entropy coding, evaluation) to improve readability, debugging, and maintainability.

4. **Experimental Analysis :**

a. **Compression Ratio vs Quantization Factor (Fq) :**

The quantisation factor Fq scales the JPEG quantisation matrix.

- A higher Fq forces more DCT coefficients to zero after rounding, increasing compression at the cost of visual quality.
- Figure 5 shows the compression ratio measured on 5 I-frames for Fq values of 1, 5, 10, 20, 30, and 50.
- The results confirm the expected monotonic increase:

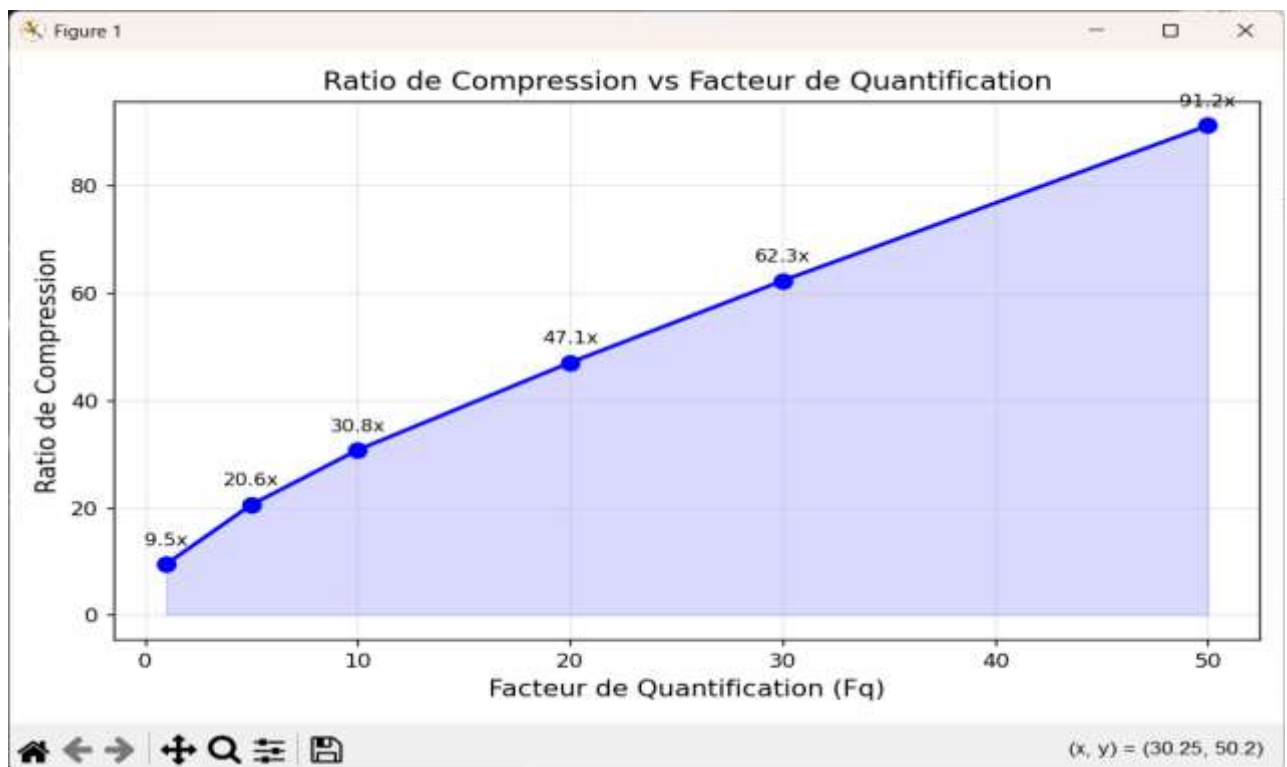


Figure 5 — Compression ratio vs Quantization Factor Fq.

b. Compression Ratio vs GOP Size :

The GOP size determines the ratio of I-frames to P-frames.

- I-frames are coded independently (higher cost), while P-frames only store motion vectors and residuals (lower cost).
- Figure 6 shows the effect of GOP size on both compression ratio and frame distribution.

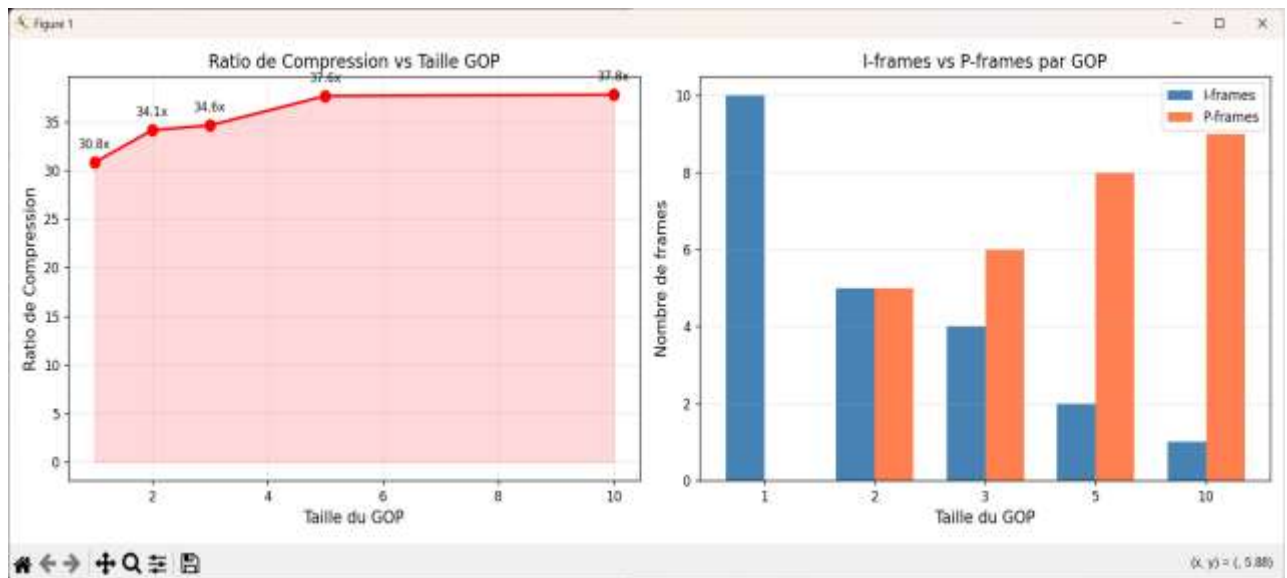


Figure 6 — Compression ratio vs GOP size and I/P-frame distribution.

5. Conclusion :

This project successfully implements and validates a complete, simplified MPEG-4-like video encoder/decoder pipeline in Python from end-to-end. Starting from raw image frames, the encoder produces a single compressed binarized file, which the decoder fully reconstructs while maintaining strong visual fidelity and high structural quality. Experimental parameter sweeps confirm the correct implementation of each modular phase, illustrating the expected monotonic behavior of the compression ratio relative to both the quantization factor and the Group of Pictures size. Ultimately, the pipeline demonstrates how spatial and temporal techniques work in harmony with entropy coding to dramatically reduce video storage requirements.